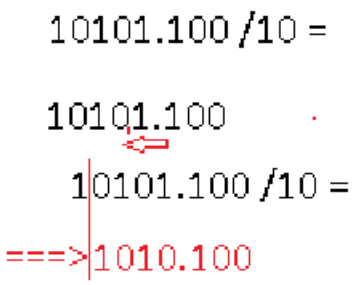
Bernal Manzanilla Saavedra IT110 B Winter 2024

Right bit shift is dividing a binary string so that the binary string/word/ number "moves" past the radix point to the right. Imagine the point remains fixed but because you are dividing by exact powers of the base. Like in decimal you shift decimal places by dividing by powers of ten (10^(-n)), similarly in binary a division by a power of two, 2^(-n) causes that the binary string/word/ number "moves" past the radix point to the right "**n**" places.

If you divide a binary number by two, the entire binary number moves one place to the right past the radix point. So, the **L**east **S**ignificant **B**it (**LSB**, to abbreviate), always becomes 1/2 or 0.5 in decimal after that division and all other bits move one place to the right turning into "one half of their initial value. That constitutes a **one bit shift to the right** by one place or that the point "floats and shifts one position to the left."



Conversely, ANY multiplication by a power of two (2^n) is a right bit shift of "n" places of a binary number/string/word, and has the **same effect than multiplying a number in decimal base (base 10) by a power of ten** (so ten one hundred, one thousand, etc.), which is causing the radix point (decimal point in base 10) to "float and shit to the right, or conversely, if the radix point remains anchored at its position, that the binary number/ word/string, shifts to the left. Thus, multiplying by **two** a binary number is the exact opposite to dividing by **two, in the sense that the** first number after the radix point becomes the **LSB and the position of all other bits shifts one place to the right past the radix point.** This last example would be equivalent to a 1-bit shift to the right past the radix point, or **one right bit shift**.

10101.100 * 10 =

10101.100

⇨

1 0101.100 * 10 =

10 101.00 <===